

Министерство науки и высшего образования РФ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Омский государственный технический университет»
Кафедра «Информатика и вычислительная техника»

ОТЧЁТ ПО РАСЧЕТНО-ГРАФИЧЕСКОЙ РАБОТЕ

По дисциплине «Арифметические и логические основы вычислительных систем»

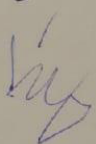
На тему:

«Умножение в прямом коде старшими разрядами вперед»

Студента Денцеля Артура Сергеевича ИВТ-233

Ассистент

Студент

 27.05.2024


А.Ю. Горшенин

А.С. Денцель

Омск 2024

Общее описание алгоритма

Регистр - устройство, в котором хранятся оперативные данные для выполнения операций. С помощью него осуществляются операции сдвига и преобразования данных, а также хранятся операнды и промежуточные данные при выполнении арифметических операций.

Сумматор – устройство, реализующее операцию сложения.

Триггер – бистабильная ячейка памяти, которая может находиться в одном из двух устойчивых состояний и способна хранить один бит информации

Предполагается, что данные на вход устройства подаются в нормализованном виде. При выполнении операции $C = A \times B$ мантисса **A** загружается в регистр **RG2**, мантисса **B** загружается в регистр **RG1**, сумматор **SM** обнуляется, в счетчик загружается число разрядов числа без учета знакового – $n-1$. При выполнении операции умножения порядки операндов должны складываться: $pC = pA + pB$, выполнение этой операции предполагается на некотором дополнительном сумматоре. Поскольку предполагается, что данные хранятся в прямом коде, то знаки операндов следует обработать отдельно и сохранить в триггере, то есть выполняется операция определения знака результата и сохранения его в триггере как

$$*T = R1[1] \oplus R2[1]$$

где $\oplus$ – логическая операция «сумма по модулю два»;

$R1[1]$ – знак B (знаковый разряд мантиссы mA);

$R2[1]$ – знак A (знаковый разряд мантиссы mB).

После чего знаки операндов необходимо принудительно обнулить.

Собственно умножение реализуется следующим образом:

1. Анализируется старший разряд регистра R1 (логическое условие $x1$), и если разряд равен 1, выполняется операция на сумматоре $SM = SM + R2$, для обеспечения данной операции в структуре предусмотрена многоразрядная связь от R2 к сумматору.
2. Выполняются сдвиг R2 в сторону младших разрядов и сдвиг R1 в сторону старших разрядов, SM остается неподвижным.

3. Содержимое счетчика уменьшается на единицу $CT--$, если $CT \neq 0$, продолжить с 1.

4. Выполняется округление $SM = SM + 2^{-(n+k)}$.

5. Если необходимо, выполняется нормализация сумматора

6. Загрузка в первый разряд сумматора знака из триггера $SM[1] = T$

Упрощенный пример выполнения арифметической операции:

Пример 1:

$$A = 0,11011100 \quad pA = 00010$$

$$B = 0,11010010 \quad pB = 00011$$

$$R2 = 0,11011100$$

$$\Sigma = 0$$

$$1) \overrightarrow{R2} = 0,01101110|0 \quad \overleftarrow{R1} = 1,\underline{1}0100100 \\ \Sigma = \Sigma + R2 = 0,01101110|0$$

$$2) \overrightarrow{R2} = 0,00110111|00 \quad \overleftarrow{R1} = 1,\underline{0}1001000$$

$$3) \overrightarrow{R2} = 0,00011011|100 \quad \overleftarrow{R1} = 0,\underline{1}0010000$$

$$\Sigma = \Sigma + R2 = 0,01101110|0 + 0,00011011|100 = 0,10001001|100$$

$$4) \overrightarrow{R2} = 0,00001101|110 \quad \overleftarrow{R1} = 1,\underline{0}0100000$$

$$5) \overrightarrow{R2} = 0,00000110|111 \quad \overleftarrow{R1} = 0,\underline{0}1000000$$

$$6) \overrightarrow{R2} = 0,00000011|011 \quad \overleftarrow{R1} = 0,\underline{1}00000000$$

$$\Sigma = \Sigma + R2 = 0,10001001|100 + 0,00000011|011 = 0,10001100|111$$

$$7) \overrightarrow{R2} = 0,00000001|101 \quad \overleftarrow{R1} = 1,\underline{0}00000000$$

$$8) \overrightarrow{R2} = 0,00000000|110 \quad \overleftarrow{R1} = 0,\underline{0}00000000$$

$$pC = pA + pB = 00101$$

$$\text{Ответ: } \Sigma = 0,10001100, pC = 00101$$

Пример 2:

$$A = 0,10100101 \quad pA = 00001$$

$$B = 0,10001010 \quad pB = 00010$$

$$R2 = 0,10100101$$

$$\Sigma = 0$$

$$1) \overrightarrow{R2} = 0,01010010|1 \quad \overleftarrow{R1} = 1,\underline{0}0010100$$

$$2) \overrightarrow{R2} = 0,00101001|01 \quad \overleftarrow{R1} = 0,\underline{0}0101000$$

$$3) \overrightarrow{R2} = 0,00010100|101 \quad \overleftarrow{R1} = 0,\underline{0}1010000$$

$$4) \overrightarrow{R2} = 0,00001010|010 \quad \overleftarrow{R1} = 0,\underline{1}0100000$$

$$\Sigma = \Sigma + R2 = 0,00001010|010$$

$$5) \overrightarrow{R2} = 0,00000101|001 \quad \overleftarrow{R1} = 1,\underline{0}1000000$$

$$6) \overrightarrow{R2} = 0,00000010|101 \quad \overleftarrow{R1} = 0,\underline{1}0000000$$

$$\Sigma = \Sigma + R2 = 0,00001010|010 + 0,00000010|101 = 0,00001100|111$$

$$7) \overrightarrow{R2} = 0,0000001|010 \quad \overleftarrow{R1} = 1,\underline{0}0000000$$

$$8) \overrightarrow{R2} = 0,0000000|101 \quad \overleftarrow{R1} = 0,\underline{0}0000000$$

$$\text{Нормализация: } \tilde{\Sigma} = 0,11000000 \quad pC = pA + pB + 1 + 1 + 1 + 1 = 00111$$

Схема устройств для работы с мантиссами и порядками

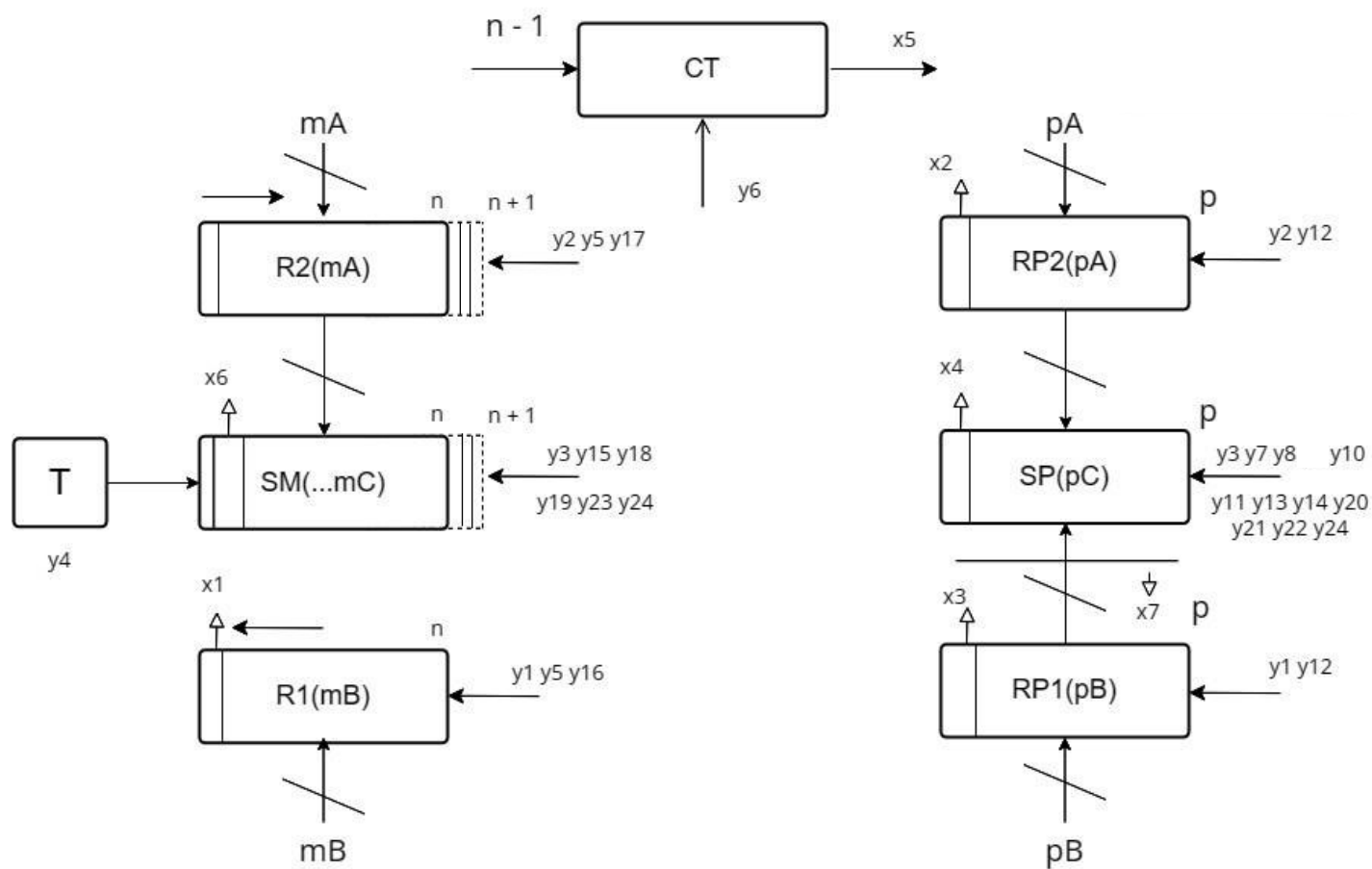


Рисунок 1. Полная структура схемы устройства умножения чисел с плавающей запятой.

Схема алгоритма работы устройства представлена на рисунках 2 и 3.

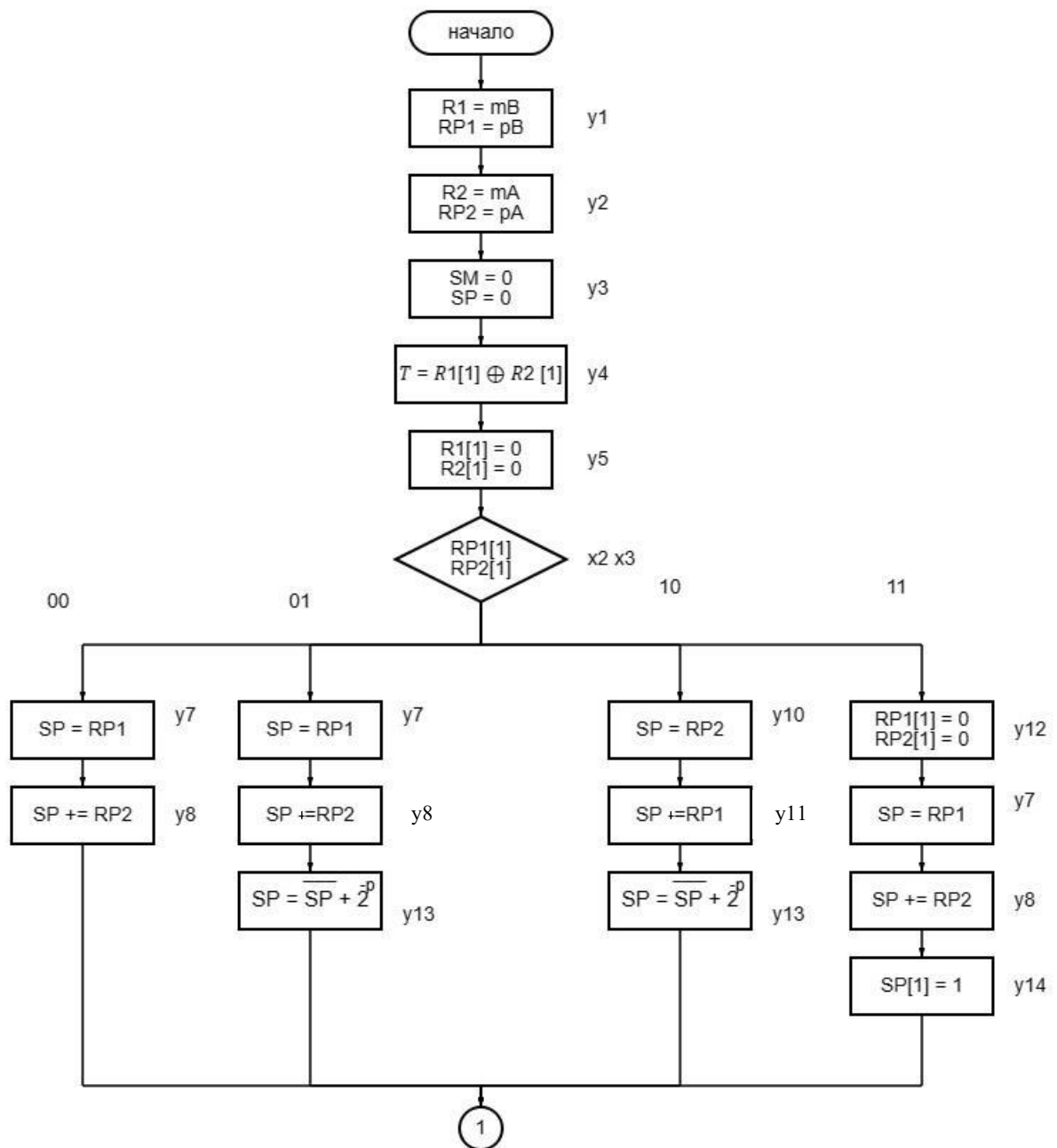


Рисунок 2. Загрузка данных с дальнейшей обработкой порядков.

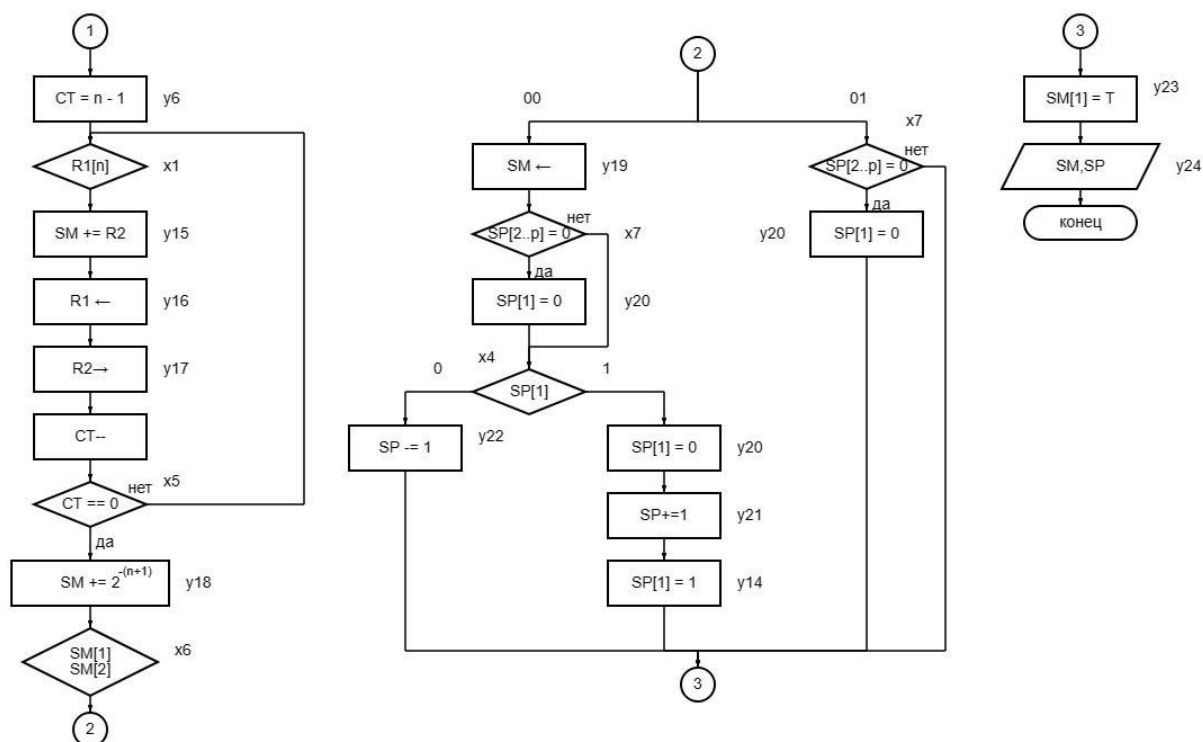


Рисунок 3. Обработка мантисс, проверка условий нормализации мантисс, вывод результирующей мантиссы и порядка.

Логические условия

Обозначение	Краткая запись	Комментарий
$x1$	$R1[n]$	Анализ старшего разряда второго множителя
$x2$	$RP1[1]$	Анализ порядка первого множителя
$x3$	$RP2[1]$	Анализ порядка второго множителя
$x4$	$SP[1]$	Знак сумматора порядков
$x5$	$CT = 0$	Проверка счетчика на ноль
$x6$	$SM[1] SM[2]$	Анализ сумматора для нормализации
$x7$	$SP[2..p]$	Проверка знака сумматора порядков на ноль

Управляющие сигналы

Обозначение	Краткая запись	Комментарий
$y1$	$R1=mB, RP1=pB$	Загрузка мантиссы и порядка В
$y2$	$R2=mA, RP2=pA$	Загрузка мантиссы и порядка А
$y3$	$SM=0, SP=0$	Сброс SM и SP

y4	$T = R1[1] \oplus R2[1]$	Получение знака результата и сохранение его в триггере
y5	$R1[1]=0 \ R2[1]=0$	Сброс знаковых разрядов R1 и R2
y6	$CT = n - 1$	Загрузка счетчика (n-1 циклов)
y7	$SP = RP1$	Загрузка сумматора SP из RP1
y8	$SP += RP2$	Сумма SP и RP2, результат в SP
y10	$SP = RP2$	Загрузка сумматора SP из RP2
y11	$SP -= RP1$	Разность SP и RP2, результат в SP
y12	$RP1[1]=0 \ RP2[1]=0$	Сброс знаковых разрядов RP1 и RP2
y13	$SP = \overline{SP} + 2^{-p}$	Перевод SP в прямой код
y14	$SP[1] = 1$	Принудительное присвоение знака результирующему порядку
y15	$SM += R2$	Сумма SM и R2, результат в SM
y16	$R1 \leftarrow$	Сдвиг R1 влево
y17	$R2 \rightarrow$	Сдвиг R2 вправо
y18	$SM += 2^{-(n+1)}$	Округление сумматора мантисс
y19	$SM \leftarrow$	Сдвиг SM влево
y20	$SP[1] = 0$	Сброс знакового разряда SP
y21	$SP += 1$	Увеличение SP на 1
y22	$SP -= 1$	Уменьшение SP на 1
y23	$SM[1] = T$	Присвоение знака результата
y24	Вывод SM, SP	Вывод мантиссы и порядка результата

Моделирование работы операционного устройства средствами языка высокого уровня(C++), разработка соответствующей программы и результаты находится в приложении 1 и 2.

Приложение 1

Программа, реализующая работу устройства умножения в прямом коде в сторону старших разрядов:

```
#include <iostream>
#include <bitset>
#include <string>

using namespace std;

int main() {
    setlocale(LC_ALL, "rus");

    unsigned char SM = 0;

    bool lastbit = false;

    unsigned char R2; // A

    cout << "Введите знаковый бит A: ";
    string zA;
    cin >> zA;

    cout << "Введите мантиссу A (8 разрядов): ";
    string mA;
    cin >> mA;
    R2 = bitset<8>(mA).to_ulong();

    unsigned char R1; // B

    cout << "Введите знаковый бит B: ";
    string zB;
    cin >> zB;

    cout << "Введите мантиссу B (8 разрядов): ";
    string mB;
    cin >> mB;
    R1 = bitset<8>(mB).to_ulong();

    unsigned char pA; // pA

    cout << "Введите порядок pA (5 разрядов): ";
    string porA;

    cin >> porA;
    pA = bitset<8>(porA).to_ulong();

    unsigned char pB; // pB

    cout << "Введите порядок pB (5 разрядов): ";
    string porB;
    cin >> porB;

    pB = bitset<8>(porB).to_ulong();

    unsigned char pC = pA + pB;

    for (int i = 0; i < 8; i++) { // Реализация операции умножения с неподвижным
        сумматором
```

```

    R2 = R2 >> 1;
    R1 = R1 << 1;
    if (R1 & 0b10000000) {
        SM += R2;
        lastbit = true;
    }
    else {
        lastbit = false;
    }
}

// Нормализация
if ((SM & 0b10000000) == lastbit) {

    while ((SM & 0b10000000) == lastbit) {
        SM = SM << 1;
        pC = pC + 1;
    }

}

// Вывод результатов
if(lastbit == true) {
    cout << "mC: " << "1."<< SM << endl;
}
else {
    cout << "mC: " << "0." << bitset<8>(SM) << endl;
}

cout << "pC: " << bitset<5>(pC);

return 0;
}

```

Приложение 2

Результаты вывода программы:

Пример 1, решенный с помощью кода:

```
Консоль отладки Microsoft Visual Studio
Введите знаковый бит A: 0
Введите мантиссу A (8 разрядов): 11011100
Введите знаковый бит B: 0
Введите мантиссу B (8 разрядов): 11010010
Введите порядок pA (5 разрядов): 00010
Введите порядок pB (5 разрядов): 00011
mC: 0.10001100
pC: 00101
```

Пример 2, решенный с помощью кода:

```
Введите знаковый бит A: 0
Введите мантиссу A (8 разрядов): 10100101
Введите знаковый бит B: 0
Введите мантиссу B (8 разрядов): 10001010
Введите порядок pA (5 разрядов): 00001
Введите порядок pB (5 разрядов): 00010
mC: 0.11000000
pC: 00111
```